Technical Documentation: Garcia Robotics Platform

Reference and Troubleshooting Guide for
Festival Nacional de Robótica – First Challenger

Gabriel Araújo, Pedro Alves, Guilherme Vidinha, Rui Meireles

ISEP – LSA

Academic Year 2025/2026

Contents

1 Hardware Architecture **3**
- 1.1 System Overview . . . 3
- 1.2 Microcontroller: Arduino Mega 2560 . . . 3
  - 1.2.1 What It Is . . . 3
  - 1.2.2 Why It Was Chosen . . . 4
  - 1.2.3 Typical Problems and Checks . . . 4
- 1.3 Locomotion and Motor Shield . . . 4
  - 1.3.1 Motor Shield and H-Bridge . . . 4
  - 1.3.2 Differential Drive . . . 4
  - 1.3.3 Typical Problems and Checks . . . 6
- 1.4 Power Supply and Safety . . . 6
  - 1.4.1 Power Layout . . . 6
  - 1.4.2 Typical Problems and Checks . . . 6
- 1.5 Environmental Sensors . . . 7
  - 1.5.1 Ultrasonic SONAR Sensors . . . 7
  - 1.5.2 Line Sensor . . . 7
  - 1.5.3 Color Sensor (TCS3200) . . . 8
- 1.6 Feedback and User Interface . . . 9
  - 1.6.1 LCD I2C . . . 9
  - 1.6.2 RGB LED and Buzzer . . . 9

2 Hardware Pinout Tables **10**

3 Software: The Garcia Library **12**
- 3.1 Purpose of the Library . . . 12
- 3.2 Initialization: `setupGarcia()` . . . 12
  - 3.2.1 What It Does . . . 12
  - 3.2.2 If Something Goes Wrong . . . 12
- 3.3 Motor-Related Functions . . . 12
  - 3.3.1 What They Do . . . 13
  - 3.3.2 Common Issues . . . 13
- 3.4 Sensor Classes . . . 13
  - 3.4.1 SONAR . . . 13
  - 3.4.2 Line Sensor . . . 13
  - 3.4.3 Color Sensor . . . 13
- 3.5 Feedback Functions . . . 14
  - 3.5.1 LCD . . . 14
  - 3.5.2 RGB LED and Buzzer . . . 14

4 Festival Nacional de Robótica – First Challenger **15**
4.1 General Overview . . . 15
4.2 Levels and What the Robot Must Do . . . 15
4.2.1 Level I – Line Following . . . 15
4.2.2 Level II – Line Following with Color Marks . . . 15
4.2.3 Level III – Sector Without Line and Lateral Wall . . . 16
4.3 Track Characteristics . . . 16
4.4 Age and Robot Categories . . . 16

5 Troubleshooting Guide **17**
5.1 General Approach . . . 17
5.2 Robot Does Not Move . . . 17
5.2.1 Symptoms . . . 17
5.2.2 Checks . . . 17
5.3 Robot Moves But Not as Expected . . . 17
5.3.1 Symptoms . . . 17
5.3.2 Checks . . . 18
5.4 Line Sensor Problems . . . 18
5.4.1 Robot Cannot Find the Line . . . 18
5.4.2 Robot Loses the Line Easily . . . 18
5.5 Color Sensor Problems . . . 18
5.5.1 Color Not Detected or Misclassified . . . 18
5.5.2 Color Changes Rapidly While Robot is Stopped . . . 18
5.6 SONAR Problems . . . 19
5.6.1 Distance Always Zero or Very Small . . . 19
5.6.2 Distance Always Very Large or Constant . . . 19
5.7 LCD, RGB LED and Buzzer Problems . . . 19
5.7.1 LCD Shows Random Characters or Nothing . . . 19
5.7.2 RGB LED Wrong Color . . . 19
5.7.3 Buzzer Too Quiet or Silent . . . 19

A Cheat Sheet C/c++ laguange **21**

Chapter 1

Hardware Architecture

1.1 System Overview

The Garcia Robotics platform is a mobile differential drive robot based on the Arduino Mega 2560 microcontroller. It was designed as an educational platform: all sensors and actuators are visible, easily accessible and connected using standard Dupont cables.

Main components:

- Arduino Mega 2560 microcontroller board.
- Motor shield based on the L298P H-bridge.
- 4 DC motors with high-traction wheels (driving a differential base).
- 5-channel infrared line sensor.
- Two ultrasonic distance sensors (sonars).
- TCS3200 color sensor module.
- RGB LED and active buzzer.
- I2C 16x2 character LCD.
- Battery pack and acrylic chassis.

This chapter explains what each component does and how it works physically, to help identify the source of problems when the robot does not behave as expected.

1.2 Microcontroller: Arduino Mega 2560

1.2.1 What It Is

The Arduino Mega 2560 is a development board that contains:

- The ATmega2560 microcontroller (the "brain").
- A USB interface for programming and serial communication with a PC.
- Voltage regulators and connectors for powering the board from USB or an external source.
- 54 digital pins, 16 analog input pins and dedicated I2C/SPI/UART pins.

1.2.2 Why It Was Chosen

The Mega was chosen instead of an Arduino Uno because:

- The Garcia robot uses many pins (motors, 5-line sensor channels, sonars, color sensor, RGB LED, buzzer, LCD).
- The extra memory and I/O allow future extensions without redesigning the base.

1.2.3 Typical Problems and Checks

- **Robot does not run any code:**
  - Check if the correct board (Arduino Mega 2560) and port are selected in the Arduino IDE.
  - Check if the USB cable is working and the board power LED is on.
- **Serial Monitor shows garbage characters:**
  - Confirm that the baud rate in the Serial Monitor matches `Serial.begin(9600);` or the value used.

1.3 Locomotion and Motor Shield

1.3.1 Motor Shield and H-Bridge

The motor shield is based on the L298P dual H-bridge. An H-bridge allows the Arduino to:

- Turn each motor on and off.
- Change the direction of rotation (forward / backward) by reversing the polarity.
- Adjust the average voltage using PWM to change speed.

The shield connects directly on top of the Arduino Mega and brings the motor terminals and screw terminals to convenient positions.

1.3.2 Differential Drive

The Garcia robot has two driven sides:

- Left side: powered by one pair of motors.
- Right side: powered by the other pair.

By commanding different speeds and directions for the left and right sides, any motion on the plane is possible (forward, backward, turning).

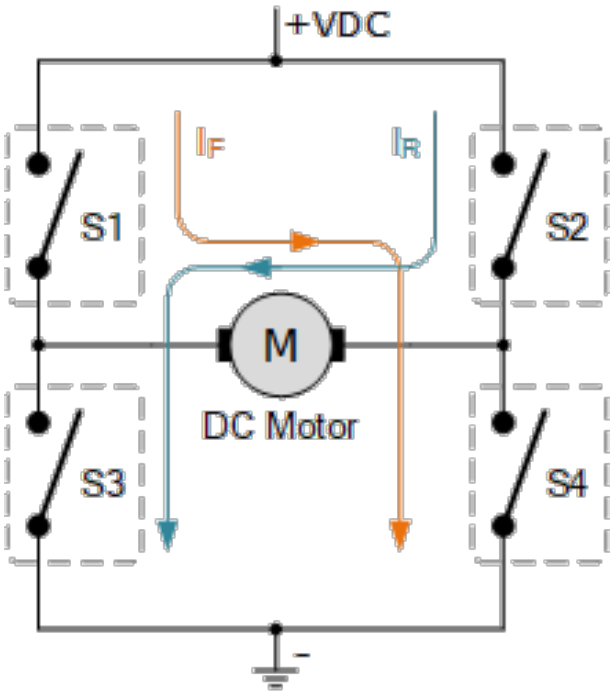


Figure 1.1: How H-bridge works

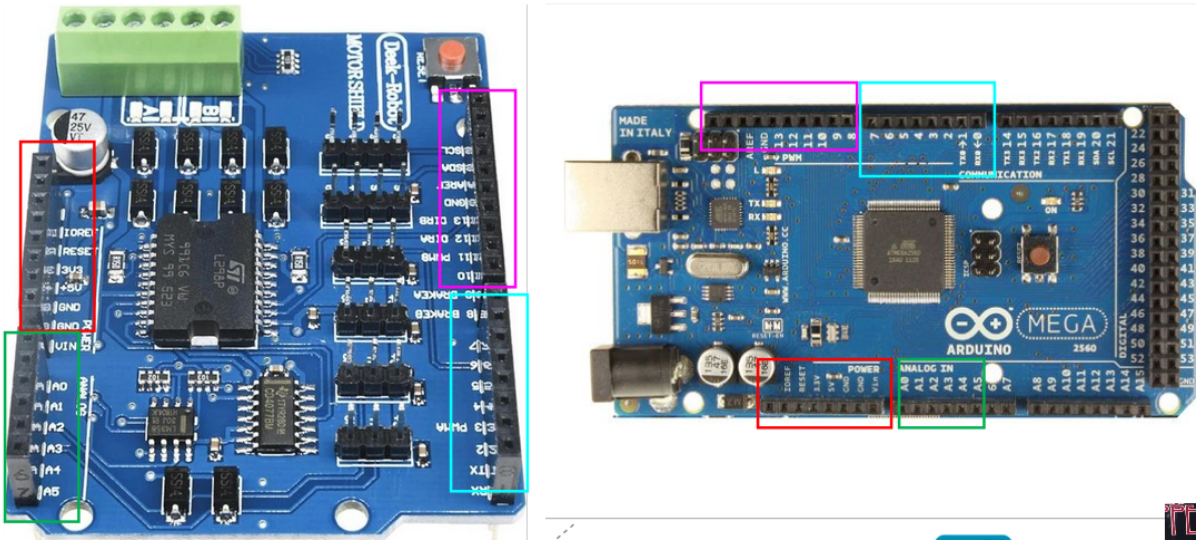


Figure 1.2: Shield connections

1.3.3 Typical Problems and Checks

- **Robot does not move at all:**
  - Check if the battery pack is connected and switched on.
  - Check the motor shield power connector and any fuses or switches.
  - Verify that the ground of the battery and Arduino are common.
- **Motors move but in the wrong direction:**
  - Check motor wiring (swap the two wires of the affected motor).
  - If using library functions like `MotorLeft(..., FORWARD)`, confirm that the library's idea of "FORWARD" matches the desired physical direction.
- **One side moves, the other does not:**
  - Check screw terminals for loose motor wires.
  - Check that both direction and PWM pins for that side are correctly soldered and not bent.

1.4 Power Supply and Safety

1.4.1 Power Layout

The Garcia platform usually uses:

- A battery pack connected to the motor shield (motors).
- The motor shield or Arduino's regulator to supply 5 V to the logic (Arduino and sensors).

The Arduino is powered by the motor shield.

1.4.2 Typical Problems and Checks

- **Robot reboots when motors start:**
  - Batteries may be low or unable to supply enough current.
  - Check for thin or long power wires causing voltage drops.
  - Ensure motor power and logic power are wired according to the recommended scheme in the assembly formation.
- **Sensors behave randomly:**
  - Noise from motors can affect sensor readings.
  - Check if power and ground wires are firmly connected and if sensor cables are not damaged.

1.5 Environmental Sensors

1.5.1 Ultrasonic SONAR Sensors

How They Work

Each ultrasonic sensor has:

- A transmitter that emits an ultrasonic pulse when TRIG is activated.
- A receiver that listens for the echo and signals its duration on ECHO.

The time between sending the pulse and receiving the echo is proportional to the distance to the obstacle.

What to Expect

- When there is a wall in front at around 20 cm, readings should be near 20 cm.
- If the sensor is pointed to open space, readings may be very large or a special "out of range" value.

Common Problems and Diagnostics

- **Distance always reads 0 or extremely small:**
  - Check wiring: TRIG and ECHO not swapped, VCC = 5 V, GND = GND.
  - Check if the sensor face is not blocked or covered by the chassis.
- **Distance always very large or constant:**
  - Verify that the surface in front is reflective to ultrasound (soft cloth may absorb sound).
  - Check if another sonar very close is interfering.

1.5.2 Line Sensor

How It Works

The line sensor has 5 infrared reflectance channels. Each channel:

- Emits IR light towards the ground.
- Measures the reflected light with a photodetector.

Black lines reflect less IR light than white background, creating a difference in the sensor output.

What to Expect

- Over white: sensor outputs are in one state (e.g., HIGH).
- Over black: outputs change to the opposite state (e.g., LOW), or analog values drop.
- When the sensor is at a constant height (a few millimeters above the surface), the transition between white and black should be clear.

Common Problems and Diagnostics

- **All channels show the same value everywhere:**
  - Check that the sensor is powered (VCC and GND).
  - Verify that the sensor is not too far from the ground.
  - Check if you are reading the correct pins in the code.
- **Sensor is extremely noisy:**
  - Check for strong ambient light directly shining on the sensors.
  - Clean the underside of the robot (dust or dirt can affect readings).

1.5.3 Color Sensor (TCS3200)

How It Works

The TCS3200 color sensor:

- Uses a matrix of photodiodes with red, green, blue and clear filters.
- Selects one filter type at a time via digital control pins S2 and S3.
- Outputs a square wave on OUT whose frequency depends on the intensity of the selected color.

The Arduino measures this frequency and interprets it as a color component value.

What to Expect

- Over a red mark: the "red" frequency should be higher than the green and blue frequencies.
- Over a green mark: green component dominates.
- Over a blue mark: blue component dominates.

Common Problems and Diagnostics

- **Sensor always reports the same color:**
  - Check the distance: the sensor must be at a consistent and relatively small distance from the surface.
  - Re-check calibration values used in the library for each color.
  - Verify that S2, S3 and OUT are connected to the correct Arduino pins.
- **Readings change randomly:**
  - Check for changing ambient light (e.g., sunlight, lamps).
  - Avoid having the sensor tilted or moving too quickly over the mark during tests.

1.6 Feedback and User Interface

1.6.1 LCD I2C

The LCD is a 16x2 character display controlled over I2C. It is useful for:

- Showing sensor readings in real time.
- Displaying simple status messages (e.g., “Ready”, “Error”, “Battery Low”).

Common Problems and Diagnostics

- **LCD does not show any characters:**
  - Check SDA/SCL connections (pins 20 and 21 on the Mega).
  - Adjust the contrast potentiometer on the LCD backpack.
  - Ensure the correct I2C address is used in the library (often 0x27 or 0x3F).

1.6.2 RGB LED and Buzzer

The RGB LED and buzzer provide visual and acoustic feedback.

What to Expect

- When each channel of the RGB LED is turned on individually, you should see red, then green, then blue light.
- When the buzzer pin is toggled or a tone function is used, a clear beep should be heard.

Common Problems and Diagnostics

- **RGB LED never lights up:**
  - Check that the common pin (anode or cathode) is wired correctly to 5 V or GND as required.
  - Verify that pins 50, 51 and 52 are correctly connected to the R, G and B legs through current-limiting resistors if required by the design.
- **Buzzer is silent:**
  - Confirm the buzzer VCC and GND connections and that the signal pin (e.g., 45) is correctly wired.
  - Test the buzzer with a very simple sketch that toggles the pin or uses `tone()`.

Chapter 2

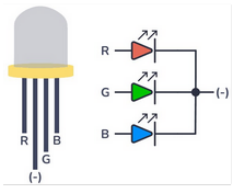
Hardware Pinout Tables

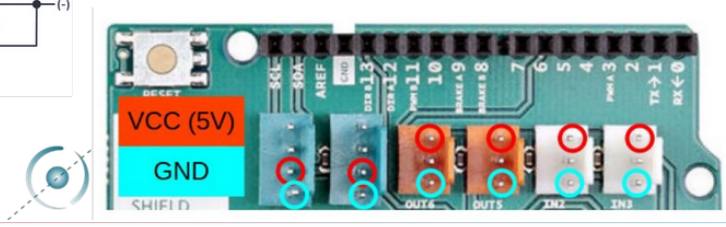
Sonar Direito	
Sensor	Arduino
VCC	5V
GND	GND
ECHO	35
TRIGGER	34

Sonar Esquerdo	
Sensor	Arduino
VCC	5V
GND	GND
ECHO	33
TRIGGER	32

Sensor de Linha	
Sensor	Arduino
VCC	5V
GND	GND
S1	36
S2	37
S3	38
S4	39
S5	40

Led RGB	
Sensor	Arduino
GND	GND
R	50
G	51
B	52



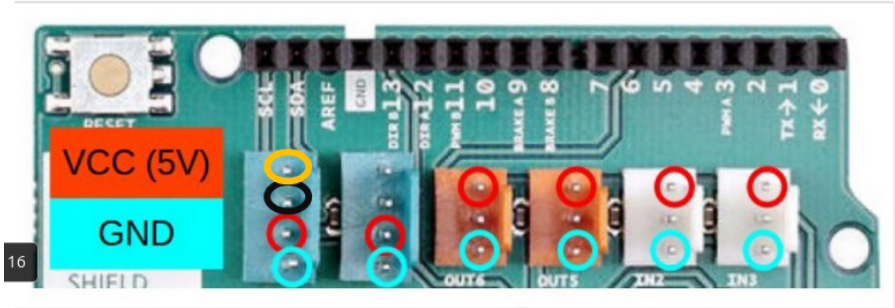


ISEP INSTITUTO SUPERIOR DE ENGENHARIA DO PORTO

LCD	
Sensor	Arduino
VCC	5V
GND	GND
SDA	SDA/20
SCL	SCL/21

Buzzer	
Sensor	Arduino
VCC	46
GND	GND

Sensor de Cor	
Sensor	Arduino
VCC	5V
GND	GND
S0	5V
S1	GND
S2	42
S3	43
OUT	44
OE	-----



INESCT TECHNOLOGY & SC

Figure 2.1: Pinout and connections

Chapter 3

Software: The Garcia Library

3.1 Purpose of the Library

The `Garcia Library` provides a set of functions and classes that:

- Hide pin numbers and low-level configuration details.
- Offer simple names that correspond to physical parts of the robot (e.g., `MotorLeft`, `sonar_right`, `color`).
- Allow quick testing and diagnosis of hardware.

This chapter explains what the key functions do and what to check if their behaviour is not what you expect. It does not describe how to design complete competition strategies. Can see the library code and more detailed implementation in Garcia Library.

3.2 Initialization: `setupGarcia()`

3.2.1 What It Does

- Configures all pin modes (inputs / outputs).
- Initializes internal objects for each sensor and actuator.
- Ensures motors are stopped and feedback devices are in a safe state at startup.

3.2.2 If Something Goes Wrong

- **Some sensors or motors do not respond at all:**
  - Check that `setupGarcia();` is actually called inside `setup()`.
  - Ensure that `garcia.h` is correctly included and the library is installed.

3.3 Motor-Related Functions

Typical functions:

- `MotorLeft(speed, direction)`

- `MotorRight(speed, direction)`
- `StopMotors()`

3.3.1 What They Do

- `MotorLeft` and `MotorRight` internally set the direction pins and apply a PWM signal to the speed pins.
- `StopMotors()` sets the PWM to zero and may also disable the H-bridge outputs.

3.3.2 Common Issues

- **Calling `MotorLeft` or `MotorRight` does nothing:**
  - Confirm that `speed` is within a valid range (usually 0–255).
  - Check power supply and motor wiring as described in the hardware chapter.

3.4 Sensor Classes

3.4.1 SONAR

Objects like `sonar_left` and `sonar_right` usually provide a method:

- `dist_cm()`: returns an estimated distance in centimeters.

Diagnostic Use

- Print the result to the Serial Monitor to confirm that values change when an obstacle is moved closer or farther away.

3.4.2 Line Sensor

The line sensor class provides methods to read the current state of the 5 channels and possibly a combined position or pattern.

Diagnostic Use

- Use a test sketch that prints raw readings from each channel.
- Move the robot slowly over the line and verify that the pattern of readings changes when crossing the line.

3.4.3 Color Sensor

The ColorSensor class encapsulates the TCS3200. Typical methods:

- `setupColor(index, r, g, b, errorMargin)` to store reference values.
- `readColor()` to measure the current color.
- `showColor()` to print raw readings.
- `isColor(RED/GREEN/BLUE)` to classify the current reading.

Diagnostic Use

- Use the provided example (`07_ColorSensor`) to:
  - Check that readings change when different colored cards are placed under the sensor.
  - Adjust calibration values if necessary.

3.5 Feedback Functions

3.5.1 LCD

High-level functions such as:

- `lcd_write("Text", row, col)`

are used to display text without dealing with I2C details.

3.5.2 RGB LED and Buzzer

Typical functions:

- `RGB_setColor(r, g, b)` to set LED color.
- A helper function (name may vary) to produce a beep on the buzzer.

These are especially useful during debugging:

- Use different colors to indicate different program states.
- Use beeps to show that a certain condition has been reached.

Chapter 4

Festival Nacional de Robótica – First Challenger

4.1 General Overview

The Festival Nacional de Robótica (FNR) is the main robotics competition in Portugal. The First Challenger competition is a junior league designed for students who are starting in robotics.

4.2 Levels and What the Robot Must Do

4.2.1 Level I – Line Following

- The track consists of a continuous black line on a white background.
- The robot must follow the line and complete 2 laps within a maximum time (typically 3 minutes).
- If the robot leaves the line, it must return to it in the correct direction within a limited time; otherwise, the attempt ends.

4.2.2 Level II – Line Following with Color Marks

- Same base line-following as Level I.
- Colored marks (red, green, blue) are placed across the track.
- When the robot passes over a colored mark, it must:
  - Stop on top of the mark.
  - Stay stopped for a specific time (e.g., 5 seconds).
  - Emit a sound for 2 seconds during the stop.
  - Turn on the RGB LED with the detected color.
- If the robot does not perform these actions correctly, penalty seconds are added.

4.2.3 Level III – Sector Without Line and Lateral Wall

- Includes all requirements from Level II.
- Contains a sector where the black line disappears and the robot must move inside a corridor with side walls.
- The robot must progress through this corridor without getting stuck or staying pressed against a wall for too long.

4.3 Track Characteristics

- Black line on white background, with curves and straight segments.
- Color marks of known size and colors, placed at specific positions.
- Corridor without line with fixed width and wall height, dimensioned for small robots like Garcia.

4.4 Age and Robot Categories

- Teams are grouped in age categories (e.g., 8–12, 13–16, 17–19 years).
- Robots are grouped into:
  - Commercial kits (e.g., LEGO).
  - Custom robots (such as Garcia, based on Arduino).

Chapter 5

Troubleshooting Guide

5.1 General Approach

When something goes wrong, it is important to:

1. Identify which subsystem is affected (motors, line sensor, color sensor, sonars, feedback, power).
2. Test that subsystem in isolation using simple code.
3. Fix wiring or configuration problems before trying to run the full competition program again.

This chapter lists common symptoms and possible causes.

5.2 Robot Does Not Move

5.2.1 Symptoms

- Upload is successful, but after reset the wheels do not turn.
- LEDs on the Arduino are on, but motors remain stopped.

5.2.2 Checks

- Battery pack switched on and properly connected.
- Motor shield securely plugged into the Arduino Mega.
- Correct code uploaded, with `setupGarcia();` called in `setup()`.
- Use a minimal motor test sketch from the Garcia examples to ensure motors respond.

5.3 Robot Moves But Not as Expected

5.3.1 Symptoms

- Robot turns when it should go straight.
- One side is clearly faster than the other.

5.3.2 Checks

- Verify that left and right motors are connected to the correct outputs on the motor shield.
- Check that tire pressures and mechanical friction are similar between sides.
- Test each side separately with constant speed commands to see if mechanical differences exist.

5.4 Line Sensor Problems

5.4.1 Robot Cannot Find the Line

- Ensure the line sensor is at the correct height (very close to the ground but not touching).
- Verify that the underside of the robot is not blocking IR light.
- Use a line sensor diagnostic example (if provided) to print readings and check that values change between black and white.

5.4.2 Robot Loses the Line Easily

- Check if the printed line on the track is thick and dark enough.
- Verify that the robot is not moving too fast for the sensor sampling rate.
- Ensure the sensor is mounted perpendicular to the direction of motion and centered.

5.5 Color Sensor Problems

5.5.1 Color Not Detected or Misclassified

- Re-run color calibration using the library example, under lighting conditions similar to the competition venue.
- Check that the sensor is not tilted and is at the recommended height above the colored mark.
- Avoid strong external light sources (sunlight directly on the sensor).

5.5.2 Color Changes Rapidly While Robot is Stopped

- Check that the robot is really stationary over the mark (no vibrations).
- Make sure the mark covers the entire field of view of the sensor.

5.6 SONAR Problems

5.6.1 Distance Always Zero or Very Small

- Confirm VCC and GND connections.
- Ensure TRIG and ECHO pins are not inverted.
- Check if the sensor is too close to the obstacle (inside the sensor minimum range).

5.6.2 Distance Always Very Large or Constant

- Try measuring distances to different materials (cardboard, wall, hand) to see if reflection improves.
- Verify no other sensors are sending ultrasonic pulses very close by.

5.7 LCD, RGB LED and Buzzer Problems

5.7.1 LCD Shows Random Characters or Nothing

- Check the I2C address used in code.
- Adjust the contrast potentiometer.
- Verify SDA and SCL are not swapped.

5.7.2 RGB LED Wrong Color

- Check that R, G and B pins correspond to the correct Arduino pins 50, 51 and 52.
- Make sure the common leg is correctly wired (to 5 V or GND according to LED type).

5.7.3 Buzzer Too Quiet or Silent

- Test with a direct simple example.
- Confirm that the buzzer has the correct polarity (if it is polarized).

Conclusion

This document describes the components of the Garcia Robotics platform, how they work and how to diagnose common problems. Its goal is to answer simple questions during development and competition preparation, especially when something does not work as expected. Designing and improving the full competition solution is left to the creativity and work of each team, using this guide as a technical reference for the underlying hardware and library behaviour.

Appendix A

Cheat Sheet C/c++ laguange

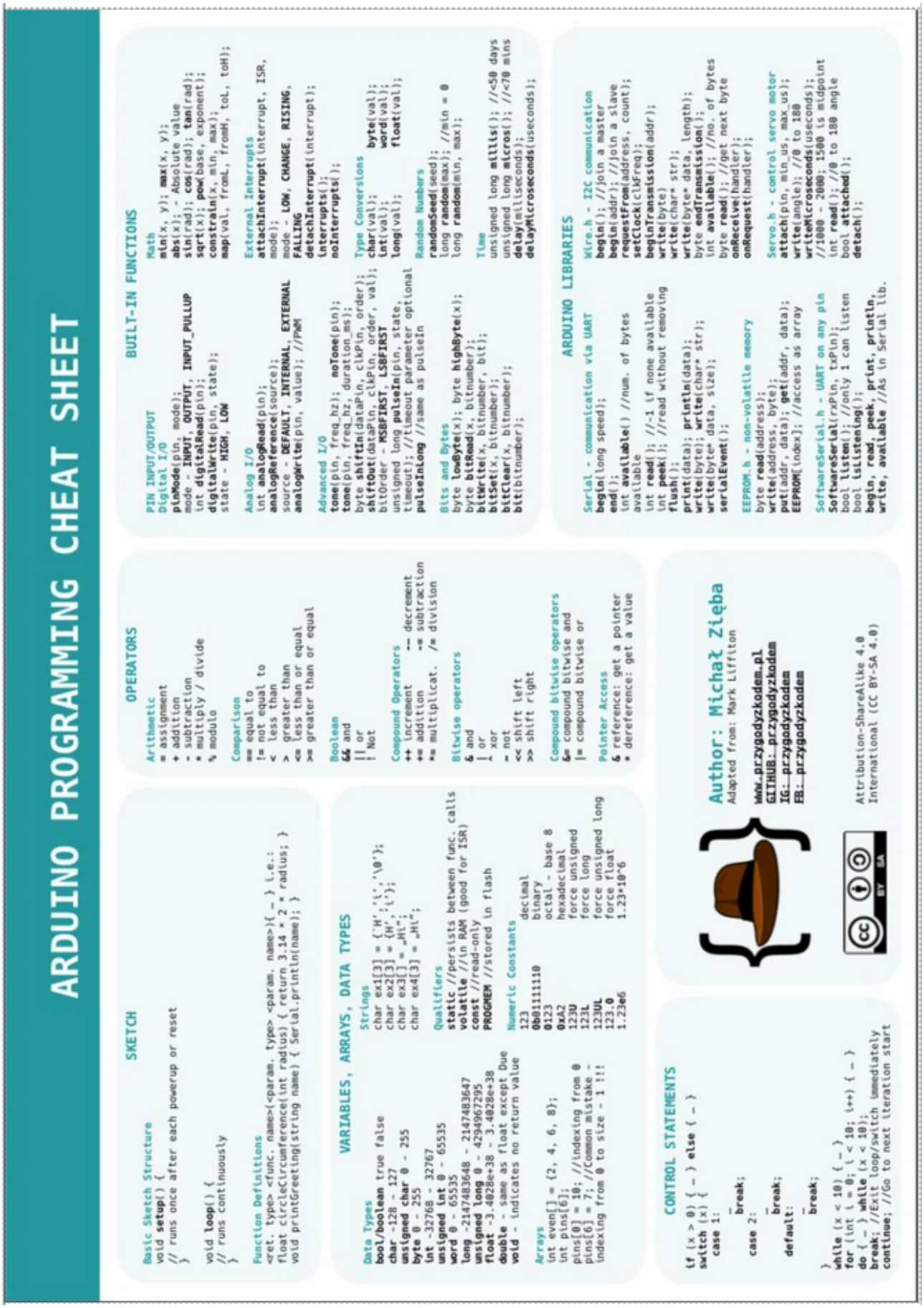


Figure A.1: Cheat Sheet

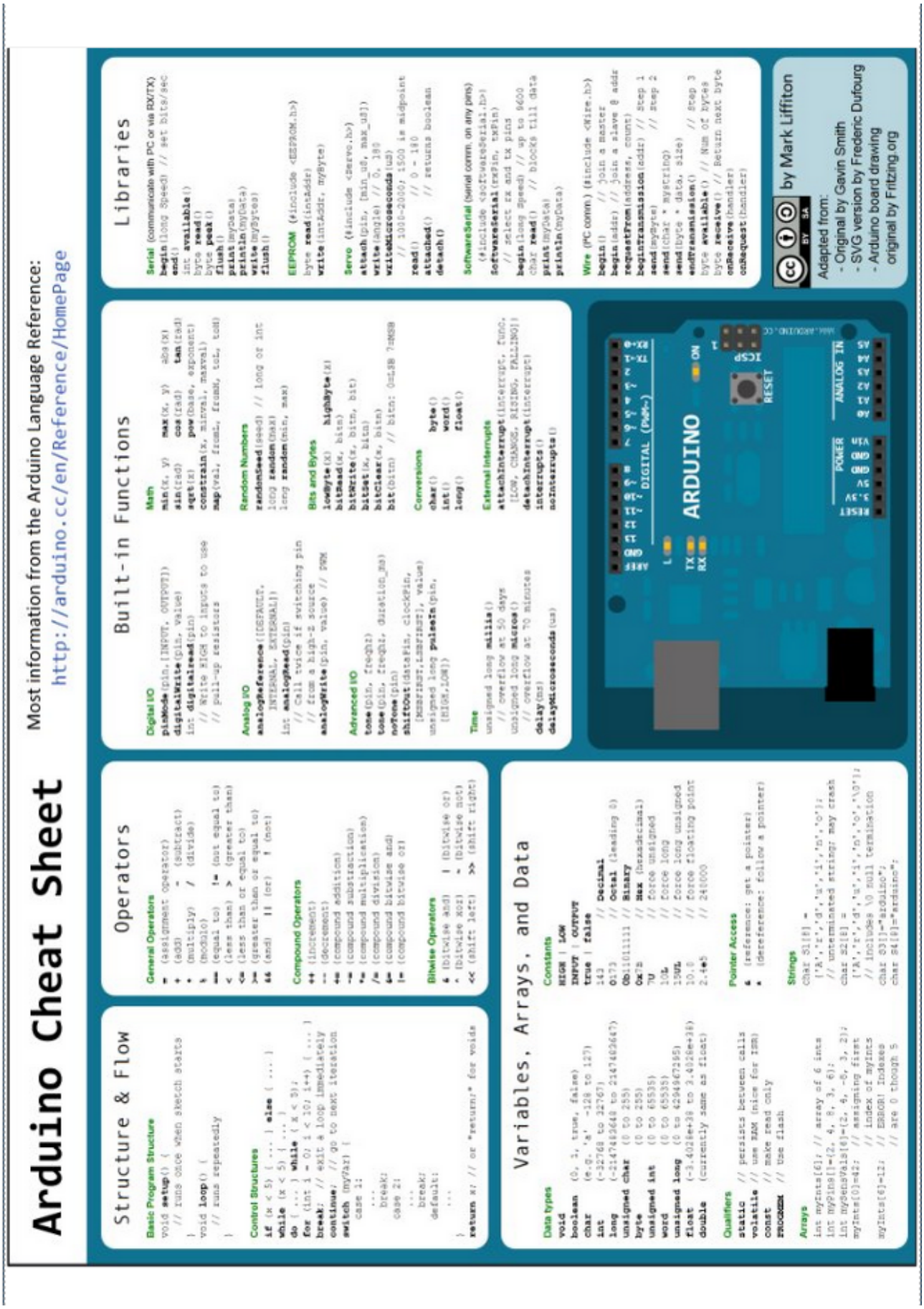


Figure A.2: Cheat Sheet 2